```
// compile_template.cpp : Defines the entry
point for the console application.
//

#include<iostream>

// template <unsigned long N>
// struct binary
/* prepend higher bits
to lowest bit */
// {
//    static unsigned const value = binary<N/
10>::value * 2 + N%10;
// };

/* specialization
terminates recursion */
// template <>
// struct binary<0>
// {
//    static unsigned const value = 0;
// };

template <unsigned long N>
struct binary
{
    // prepend higher bits
    // to lowest bit
    enum {value = binary<N/10>::value * 2 + N %
10};
};

template <>
// specialization
struct binary<0>
1// terminates recursion
{
       enum {value = 0};
};


// this class encapsulates the "+" operation.
struct plus{
    static double apply( double a, double b)
    {
        return a+b;
    }
};

// the node in the parse tree.
template <typename Left, typename Op, typename
Right>
struct X{
    Left    left;
    Right   right;

    X( Left t1, Right t2) : left(t1), right(t2)
{ }

    double operator[](int i){
        return Op::apply( left[i], right[i] );
    }
};

struct Array{
    // constructor
    Array( double *data, int N) : data_(data),
N_(N) { }

    // assign an expression to the array
    template <typename Left, typename Op,
typename Right>
    void operator=( X<Left,Op,Right> expr){
        for ( int i = 0; i < N_; ++i)
            data_[i] = expr[i];
    }

    double operator[](int i){
        return data_[i];
    }

    double *data_;
    int N_;
};

template <typename Left>
X<Left, plus, Array> operator+( Left a, Array
b){
    return X<Left, plus, Array>(a,b);
}

int main() {
    const unsigned int bi1 =
binary<110011001111>::value;
    std::cout<< bi1<<std::endl;

    double a_data[] = { 2, 3, 5, 9 };
    double b_data[] = { 1, 0, 0, 1 };
    double c_data[] = { 3, 0, 2, 5 };
    double d_data[4];

    Array A(a_data,4);
    Array B(b_data,4);
    Array C(c_data,4);
    Array D(d_data,4);

    D = A + B + C;

    for ( int i = 0; i < 4; ++i ) std::cout <<
D[i] << " "; std::cout << std::endl;
    return 0;
}

--------

3279
6 3 7 15
```

```
//constexpr C++11
//>=msvs 2015
#include <iostream>

using namespace std;

// Pass by value
constexpr float pow(float x, int n)
{
    return n == 0 ? 1 :
        n % 2 == 0 ? pow(x * x, n / 2) :
        pow(x * x, (n - 1) / 2) * x;
};

// Pass by reference
constexpr float pow2(const float& x, const int&
n)
{
    return n == 0 ? 1 :
        n % 2 == 0 ? pow2(x * x, n / 2) :
        pow2(x * x, (n - 1) / 2) * x;
};

// Compile time computation of array length
template<typename T, int N>
constexpr int length(const T(&ary)[N])
{
    return N;
}

// Recursive constexpr function
constexpr int fac(int n)
{
    return n == 1 ? 1 : n*fac(n - 1);
}

// User-defined type
class Foo
{
public:
    constexpr explicit Foo(int i) : _i(i) {}
    constexpr int GetValue()
    {
        return _i;
    }
private:
    int _i;
};

int main()
{
    //foo is const (only in C++11):
    constexpr Foo foo(5);
    // foo = Foo(6); //Error!

    //Compile time:
    constexpr float x = pow(5, 3);
    constexpr float y { pow(2, 5) };
    constexpr int val = foo.GetValue();
    constexpr int f5 = fac(5);
    const int nums[] { 1, 2, 3, 4 };
    const int nums2[length(nums) * 2] { 1, 2,
3, 4, 5, 6, 7, 8 };

    //Run time:
    cout << x << endl;
    cout << y << endl;
    cout << f5 << endl;
    cout << "The constexpr value of foo is " <<
val << endl;
    cout << "The value of foo is " <<
foo.GetValue() << endl;
    cout << "length(nums) = " << length(nums)
<< endl;
    cout << "length(nums2) = " << length(nums2)
<< endl;

}
```

```
-------

125
32
120
The constexpr value of foo is 5
The value of foo is 5
length(nums) = 4
length(nums2) = 8

-------
```

```
//constexpr c++14
//>=msvs 2017

#include <iostream>

using namespace std;

constexpr int my_strcmp( const char* str1,
const char* str2 ) {
    int i = 0;
    for( ; str1[i] && str2[i] && str1[i] ==
str2[i]; ++i )
        { }
    if( str1[i] == str2[i] ) return 0;
    if( str1[i] < str2[i] ) return -1;
    return 1;
}

constexpr const char* mama = "mama";
constexpr const char* mama2 = "mama";
constexpr const char* mamb = "mamb";
constexpr const char* mam0 = "mam0";

int main()
{
    //Run time:
    cout << my_strcmp(mama, mama2) << endl;
    cout << my_strcmp(mama, mamb) << endl;
    cout << my_strcmp(mama, mam0) << endl;
}
```

```
-------

0
-1
1
```